

AI Vendor, Procurement & Inventory Platform

A focused system for a small multi-outlet cloud kitchen

IN ONE SENTENCE

It keeps vendors, purchase orders, deliveries, stock, bills and payments in one place — and then tells the kitchen **what to order, how much, from whom, and by when.**

THE PROBLEM IT REPLACES

- Vendor rates live in WhatsApp, orders in a notebook, bills in a drawer, stock in someone's head.
- Payments get missed, and vendors who are always late are never actually compared.
- Stock runs out mid-service, or sits until it spoils. Nobody knows which, until it happens.
- Nobody records how much material was *used* — so nothing can be predicted.

THE ONE IDEA THAT MAKES THE AI POSSIBLE

Dishes sold × recipe = raw material consumed

Usage is never typed in by hand. Sales data becomes consumption data automatically through the recipe — and consumption is the one number every prediction depends on, whether it is reorder timing, stockout risk or deadstock.

THE TWO LOOPS THE SYSTEM RUNS ON

Vendor rate, lead time, terms	Purchase order what & how much	Delivery received or short	Stock goes up	Bill due date set	Payment recorded
Sales imported dish × qty × day	Recipe applied material per plate	Stock goes down	Usage rate known per day, per outlet	AI advises order / transfer / stop	Owner approves becomes a PO

Top row is material and money coming in. **Bottom row** is material going out, which is what teaches the system. The two rows meet at stock.

1 System architecture

Four layers, one backend. Each layer only does what it is genuinely good at, and the boundary between them is the most important decision in the whole design.

Browser — React dashboard

Dashboard · Vendors · Purchase Orders · Stock · Recipes · Finance · AI Insights · Assistant

▼ HTTPS + login token

Backend — FastAPI (Python)

one service, four internal layers

1 Core logic — plain code, no AI

Stock balances, PO totals, bill due dates, outstanding amounts, order status. These must be exactly right, so a model is never allowed near them.

2 Analytics — formulas, not machine learning

Daily usage rate, days of cover, real vendor lead time, delay rate, reorder point, safety stock, deadstock, overstock, wastage variance. Every figure here is explainable line by line.

3 Forecast — the only real model

Fourteen-day demand per material, per outlet. Always checked against a simple baseline; if the model cannot beat the baseline, the baseline is used and that fact is shown openly.

4 Claude — explains and answers

Turns the computed numbers into a plain sentence, and answers typed questions by calling a fixed set of safe, read-only functions. It never calculates and never writes to the database.



PostgreSQL

Supabase

All business data. Stock is stored as a list of movements, never as one editable number.

Claude API

Anthropic

Sonnet for the question-answering assistant, Haiku for short cached explanations.

Nightly job

Cron

Recomputes analytics, runs the forecasts, and generates the day's recommendations.

THE RULE THAT HOLDS THE DESIGN TOGETHER

Layers 1 and 2 decide. Layer 4 only speaks. Every recommendation is produced by ordinary code, with its numbers attached to it. Claude is then handed those numbers and asked to write one sentence. That is why the advice can always be audited, and why a wrong answer traces back to a formula rather than to a model's mood.

2 Technology

LAYER	CHOICE	WHY THIS ONE
Frontend	React + Vite + TypeScript	Shared filters and an outlet switcher; no public pages to server-render.
Backend	FastAPI (Python 3.12)	Python is needed anyway for analytics and forecasting.
Database	PostgreSQL (Supabase)	Deeply related, money-sensitive data; every metric is a SQL aggregation.
Login	Supabase Auth (JWT)	Never hand-roll auth; role checked on every request.
Analytics & forecast	pandas, statsmodels, scikit-learn	History is short, so simple methods beat complex ones.
AI	Claude API	Sonnet answers questions; Haiku writes cached explanations.
Files	Supabase Storage	Bill photos and generated PO PDFs.
Hosting	Vercel + Render	Free tiers, deploy on push; Render runs the nightly job.

Cost Every service has a free tier that fits this scale. The only real spend is Claude API usage, kept small by caching explanations.

3 Three decisions worth defending

1. Consumption is *derived* from sales and recipes, never typed in — otherwise nobody records it, and the AI has nothing to work with.
2. Stock is always the *sum* of its movements, never a stored number — stored quantities drift away from their own history.
3. The model *explains and retrieves; it never calculates* — so the numbers stay correct and the advice stays traceable.